

GUIDE COMPLET DU PROJET AIDLY

Plateforme d'Services entre Particuliers

TABLE DES MATIÈRES

1. Aperçu du projet
 2. Répartition des rôles
 3. Configuration initiale
 4. Noms des branches et comment travailler
 5. Architecture du projet
 6. Livrables de chacun
 7. Dépendances entre les rôles
 8. Comment tester sans dépendre des autres
 9. Processus de fusion (Merge)
 10. Questions / Résolution de problèmes
-

APERÇU DU PROJET

Nom: Aidly

Description: Une plateforme mobile pour demander et offrir des services locaux (bricolage, cours, ménage, etc.)

Tech Stack:

- Flutter (UI)
- Firebase (Auth + Firestore)
- Provider (Gestion d'état)
- API OpenStreetMap (Localisation)

Repaires GitHub: <https://github.com/EREN762/Aidly>

RÉPARTITION DES RÔLES

RÔLE 1 – BACKEND / AUTH / MODELS (Lead Backend)

Responsable: (EVRIL)

Mission

- Gérer toute l'authentification Firebase
- Définir les modèles utilisés par toute l'app
- Gérer la base de données Firestore
- Assurer la persistance de connexion
- Implémenter la logique métier backend

Responsabilités concrètes

✓Inscription (Email + Password)

- ✓Connexion (Email + Password)
- ✓Connexion avec Google
- ✓Déconnexion
- ✓Mot de passe oublié
- ✓Persistance de connexion
- ✓Créer/Modifier/Supprimer un service dans Firestore
- ✓Récupérer la liste des services

Branche Git

```
git checkout -b backend-auth
```

Fichiers que tu touches

```
lib/
├── main.dart (initialisation Firebase)
├── models/
│   ├── user_model.dart
│   └── service_model.dart
├── services/
│   ├── auth_service.dart          (CONTRAT - les autres l'utilisent)
│   ├── firebase_auth_service.dart (Implémentation)
│   └── service_firestore.dart      (CRUD Firestore)
└── controllers/
    ├── auth_controller.dart
    └── service_controller.dart
```

RÔLE 2 – UX / DESIGN / SCREENS

Responsable: (YANICE)

Mission

- Créer tous les écrans visuels
- Garantir une UI/UX fluide et agréable
- Gérer les feedback utilisateur (messages, loading, erreurs)

Responsabilités concrètes

- ✓Écran de Login
- ✓Écran d'Enregistrement
- ✓Écran d'Accueil (liste des services)
- ✓Écran de Détail d'un service
- ✓Écran Ajouter/Modifier un service
- ✓Écran de Profil utilisateur
- ✓Gestion Loading (spinners)
- ✓Gestion Erreurs (SnackBar)

✓Gestion Confirmations (Dialogs)

Branche Git

```
git checkout -b ui-screens
```

Fichiers que tu touches

```
lib/  
└─ views/  
    │ login_screen.dart  
    │ register_screen.dart  
    │ home_screen.dart  
    │ service_detail_screen.dart  
    │ add_service_screen.dart  
    │ profile_screen.dart  
    └─ widgets/           (composants réutilisables)
```

Important:

- Tu n'appelles JAMAIS Firebase directement
- Tu utilises uniquement les Providers (fournis par Rôle 3)
- Tu affines les modèles si besoin (discussion avec Rôle 1)

RÔLE 3 – STATE MANAGEMENT & NAVIGATION

Responsable: (PROVIDI)

Mission

- Faire le pont entre UI et Backend
- Gérer l'état de l'app (Providers)
- Gérer la navigation et les routes
- Protéger les routes (login required)

Responsabilités concrètes

- ✓Créer AuthProvider (expose les fonctions d'auth)
- ✓Créer ServiceProvider (expose les fonctions Firestore)
- ✓Gérer la navigation (materialApp routes)
- ✓Rediriger vers Login si pas connecté
- ✓Décider quel écran afficher selon l'état

Branche Git

```
git checkout -b state-management
```

Fichiers que tu touches

```
lib/
├── app.dart                (Application principale)
├── providers/
│   ├── auth_provider.dart (Appelle AuthService)
│   └── service_provider.dart (Appelle ServiceFirestore)
└── routes/
    └── app_routes.dart     (Définition des routes)
```

Conseil: Attends que Rôle 1 finisse les services avant de commencer. Utilise des FakeProviders en attendant.

RÔLE 4 – API REST & LOCALISATION

Responsable: (BENJAMIN)

Mission

- Intégrer une API externe pour enrichir l'app
- Gérer la localisation et la recherche de lieux

Responsabilités concrètes

- ✓ Utiliser l'API OpenStreetMap (Nominatim)
- ✓ Rechercher une adresse par texte
- ✓ Retourner latitude/longitude
- ✓ Afficher le lieu sur une carte (optionnel)

Branche Git

```
git checkout -b external-api
```

Fichiers que tu touches

```
lib/
└── api/
    └── location_api.dart (Appels à OpenStreetMap)
```

Exemple fonction:

```
Future<LocationModel> searchLocation(String query) async {  
  // Appel API OpenStreetMap  
  // Retourner lat, lng, adresse  
}
```

RÔLE 5 – QUALITÉ / TESTS / DOCUMENTATION

Responsable: (DANIELITO)

Mission

- Vérifier que tout fonctionne ensemble
- Tester CRUD complet
- Préparer la défense orale
- Documenter le projet

Responsabilités concrètes

- ✓ Tester l'authentification complète
- ✓ Tester CRUD des services
- ✓ Vérifier l'intégration API
- ✓ Vérifier la navigation
- ✓ Rédiger/Finaliser le README
- ✓ Préparer des réponses du prof
- ✓ Vérifier l'architecture MVC

Branche Git

```
git checkout -b quality-docs
```

Fichiers que tu touches

— README.md	(Documentation principale)
— CHECKLIST_FINAL.md	(Checklist avant soutenance)
— PLANNING.md	(Planning du projet)

CONFIGURATION INITIALE

Étape 1 – Cloner le projet (TOUS)

```
git clone https://github.com/EREN762/Aidly  
cd Aidly
```

Étape 2 – Installer les dépendances (TOUS)

```
flutter pub get
```

Étape 3 – Créer votre branche personnelle (CHACUN)

Rôle 1 (Backend/Auth):

```
git checkout -b backend-auth
```

Rôle 2 (UI/Design):

```
git checkout -b ui-screens
```

Rôle 3 (State Management):

```
git checkout -b state-management
```

Rôle 4 (API/Localisation):

```
git checkout -b external-api
```

Rôle 5 (Qualité/Docs):

```
git checkout -b quality-docs
```

Étape 4 – Publier votre branche sur GitHub (CHACUN)

```
git push origin nom-de-votre-branche
```

C'est fait ! Vous pouvez commencer à travailler.

NOMS DES BRANCHES ET COMMENT TRAVAILLER {#git}

Noms des branches (OBLIGATOIRE)

Rôle	Nom de branche
------	----------------

Backend/Auth	backend-auth
UI/Design	ui-screens
State Management	state-management
API/Localisation	external-api
Qualité/Docs	quality-docs

✗ À NE JAMAIS FAIRE

- ✗ Coder directement dans `main`
- ✗ Fusionner sans tester
- ✗ Modifier des fichiers d'autres personnes sans discussion
- ✗ Renommer les fonctions/services sans prévenir

✓ WORKFLOW À SUIVRE (CHAQUE JOUR)

Avant de commencer

```
git pull origin nom-de-votre-branche
```

Faire vos changements

```
# Éditer vos fichiers
# Tester localement
```

Commit clair (TRÈS IMPORTANT)

```
git add .
git commit -m "feat: ajouter la connexion Google"
# OU
git commit -m "fix: corriger la persistance de connexion"
# OU
git commit -m "docs: ajouter documentation auth"
```

Format des commits: `feat:` pour une

- nouvelle fonctionnalité `fix:` pour une
- correction `docs:` pour la documentation
- `refactor:` pour réorganiser du code
- **Envoyer vos changements**

```
git push origin nom-de-votre-branche
```

Quand c'est fini (Pull Request)

Allez sur <https://github.com/EREN762/Aidly>

Créez une Pull Request de votre branche vers **main**

Attendez la validation du groupe

ARCHITECTURE DU PROJET {#architecture}

```
lib/
├── main.dart                # Point d'entrée (Firebase init)
├── app.dart                 # App principale avec Providers
├── models/                  # (Rôle 1)
│   ├── user_model.dart
│   └── service_model.dart
├── services/                # (Rôle 1)
│   ├── auth_service.dart   # CONTRAT d'authentification
│   ├── firebase_auth_service.dart # Implémentation Firebase
│   └── service_firestore.dart # CRUD Firestore
├── controllers/             # (Rôle 1)
│   ├── auth_controller.dart
│   └── service_controller.dart
├── providers/                # (Rôle 3)
│   ├── auth_provider.dart
│   └── service_provider.dart
├── views/                   # (Rôle 2)
│   ├── login_screen.dart
│   ├── register_screen.dart
│   ├── home_screen.dart
│   ├── service_detail_screen.dart
│   ├── add_service_screen.dart
│   ├── profile_screen.dart
│   └── widgets/             # Composants réutilisables
│       ├── service_card.dart
│       └── loading_widget.dart
├── routes/                  # (Rôle 3)
│   └── app_routes.dart
├── api/                     # (Rôle 4)
│   └── location_api.dart
└── constants/
    └── app_constants.dart
```

LIVRABLES DE CHACUN

Rôle 1 – Backend/Auth (PRIORITÉ 1)

À livrer OBLIGATOIREMENT: `user_model.dart`

- ☐ finalisé `service_model.dart` finalisé
- ☐ `auth_service.dart` (contrat complet)
- ☐ `firebase_auth_service.dart` (fonctionnel)
- ☐ `service_firestore.dart` (fonctionnel) `main.dart`
- ☐ avec Firebase init Fonctions testables en local
- ☐
- ☐ **Date limite:** Jour 2-3 (c'est la base de tout)
- ☐

Rôle 2 – UI/Screens (DÉPEND DE RÔLE 1)

À livrer OBLIGATOIREMENT:

- ☐ Tous les écrans (6 minimum)
- ☐ Composants réutilisables
- ☐ Loading, SnackBars, Dialogs
- ☐ Navigation fluide
- ☐ Pas d'appels Firebase directs

Date limite: Jour 4-5

Rôle 3 – State Management (DÉPEND DE RÔLE 1)

À livrer OBLIGATOIREMENT:

- ☐ `auth_provider.dart` fonctionnel
- ☐ `service_provider.dart` fonctionnel
- ☐ `app.dart` complet `app_routes.dart`
- ☐ avec protections Navigation Login →
- ☐ Home fonctionnelle

Date limite: Jour 3-4

Rôle 4 – API/Localisation (INDÉPENDANT)

À livrer OBLIGATOIREMENT:

- ☐ `location_api.dart` fonctionnel
- ☐ Fonction de recherche de lieu
- ☐ Parsing JSON correct
- ☐ Documentation API utilisée

Date limite: Jour 5

Rôle 5 – Qualité/Docs (DERNIER)

À livrer OBLIGATOIREMENT:

- ☐ README.md complet
- ☐ CHECKLIST_FINAL.md
- ☐ Tests globaux complétés
- ☐ Tous les livrables vérifiés

Date limite: Jour 6 (avant soutenance)

DÉPENDANCES ENTRE LES RÔLES

Rôle 1 (Backend/Auth)

↓

├→ Rôle 2 (UI) : utilise models + services

├→ Rôle 3 (State) : appelle les services

└→ Rôle 4 (API) : peut enrichir les models

↓

└→ Rôle 5 (Qualité) : teste tout ensemble

Ordre de priorité:

1. **Rôle 1** : Doit être prêt en premier (tout dépend)
2. **Rôle 3** : Peut commencer après Rôle 1
3. **Rôle 2** : Peut commencer en parallèle avec Rôle 3
4. **Rôle 4** : Indépendant, peut commencer quand veut
5. **Rôle 5** : En dernier, quand les autres ont fini

COMMENT TESTER SANS DÉPENDRE DES AUTRES

Rôle 1 – Comment tester l'Auth sans UI

Methode 1 : Écran de test temporaire

```
// lib/test_screen.dart (supprimé après)
class AuthTestScreen extends StatelessWidget {
  final authService = FirebaseAuthService();

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("Auth Test")),
      body: Column(
        children: [
          ElevatedButton(
            onPressed: () async {
              final user = await authService.signInWithEmail(
                "test@email.com",
                "password123",
              );
              print("LOGIN: ${user?.email}");
            },
            child: Text("Test Login"),
          ),
        ],
      ),
    );
  }
}
```

Methode 2 : Tests dans main() (ultra simple)

```
// Dans main.dart pour tester rapidement
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp();

  final auth = FirebaseAuthService();

  // Test direct dans la console
  auth.authStateChanges().listen((user) {
    print("User: ${user?.email}");
  });

  await auth.signInWithEmail("test@mail.com", "123456");
}
```

Rôle 2 – Comment tester l'UI sans Backend

Utiliser des **FakeProviders** :

```
// Utiliser un provider fake pendant que Rôle 1 travaille
class FakeAuthProvider extends ChangeNotifier {
  Future<void> signIn(String email, String password) async {
    // Simule une connexion
  }
}

// Dans app.dart, utiliser FakeAuthProvider au lieu du vrai
providers: [
  ChangeNotifierProvider(create: (_) => FakeAuthProvider()),
]
```

Rôle 3 – Comment tester State Management

```
// Créer des mock data
final mockUser = UserModel(
  id: "123",
  name: "Test User",
  email: "test@mail.com",
);

final mockServices = [
  ServiceModel(
    id: "1",
    title: "Bricolage",
    description: "Réparation fenêtre",
    category: "Bricolage",
    location: "Kinshasa",
    ownerId: "789",
    createdAt: DateTime.now(),
  ),
];
```

Rôle 4 – Comment tester l'API

```
// Tester directement l'API dans main()
void main() async {
  final locationApi = LocationAPI();
  final result = await locationApi.searchLocation("Kinshasa");
  print(result);
}
```

PROCESSUS DE FUSION (MERGE)

Comment fusionner votre code vers main

Étape 1 – Créer une Pull Request

1. Allez sur <https://github.com/EREN762/Aidly>
2. Cliquez sur "Pull requests"
3. Cliquez sur "New pull request"
4. Sélectionnez votre branche
5. Décrivez ce que vous avez fait

Étape 2 – Validation

- Au moins 1 autre membre doit vérifier
- Assurer-vous que l'app compile
- Assurer-vous qu'y'a pas de conflits

Étape 3 – Merge

Cliquez sur "Merge pull request"

EN CAS DE CONFLIT

Si Git dit "Conflict", c'est normal. C'est pas grave, ça veut dire :

- Vous avez modifié la même ligne
- Git ne sait pas quel version garder

Solution simple:

```
git pull origin main
# Résoudre les conflits dans l'éditeur
git add .
git commit -m "fix: resolve merge conflict"
git push origin nom-de-votre-branche
```

QUESTIONS / RÉOLUTION DE PROBLÈMES

Q1 : Mon code va disparaître quand quelqu'un d'autre push ?

Réponse: Non, jamais. Git additionne les changements. Sauf si :

- Vous modifiez exactement les mêmes lignes → conflit (Git vous prévient)
Vous forcez un push → mauvaise idée

Q2 : Comment savoir qui a changé quoi ?

Réponse: Via Git :

```
git log --oneline           # Voir tous les commits
git log --author="Nom"     # Commits d'une personne
git diff main nom-de-branche # Voir les changements
```

Q3 : Mon app ne compile pas après un merge

Réponse: Généralement, c'est un import manquant ou un nom changé.

1. Vérifiez que les imports sont bons
2. Vérifiez que les noms des fonctions n'ont pas changé
3. Demandez à Rôle 1 si quelque chose a changé

Q4 : J'ai cassé quelque chose, comment revenir en arrière ?

Réponse:

```
git revert HEAD           # Annuler le dernier commit
# OU
git reset --hard origin/nom-de-branche # Revenir à la dernière version
```

Q5 : Comment on teste si tout marche ensemble ?

Réponse:

1. Rôle 1 finit son travail
2. Rôle 3 fait le Provider
3. Rôle 2 fait les écrans
4. Rôle 5 teste la chaîne complète

En attendant : utilisez des **Fake Implementations**.

Q6 : On peut modifier les modèles pendant qu'on travaille ?

Réponse: Non ☐

- Rôle 1 les fixe une fois
- Si besoin : discussion + commit ensemble Sinon :
- conflit garanti pour tout le monde

Q7 : Comment on gère les dépendances (packages) ?

Réponse: Ajouter dans `pubspec.yaml` une seule fois :

```
flutter pub add firebase_core
# OU manuellement dans pubspec.yaml
```

Puis tout le monde fait :

flutter pub get

Q8 : Aidly c'est pour quelle région / quelle langue ?

Réponse:

- Région : Locale (ex: Kinshasa, RDC)
- Langue : Français + localisation locale OK
- Devises : Configurable

Q9 : Est-ce qu'on peut travailler la nuit et le jour ?

Réponse: Oui ! Tant que :

- Vous finissez avant de push
- Vous pushez clairement vos commits
- Vous pullez avant de commencer Q10 :

Comment préparer la soutenance ?

Réponse:

1. Rôle 5 teste tout
2. Chacun prépare son explication (2-3 min)
3. Filmer une démo
4. Préparer les réponses à "Comment vous avez travaillé en groupe ?"

RÉPONSES PARFAITES POUR LE PROF

Q: "Comment vous avez organisé le travail en groupe ?"

R: "Nous avons eu 5 rôles distincts pour éviter les conflits.
Rôle 1 a défini les modèles et services comme contrat. Les autres ont attendu, puis ont utilisé ces services via des Providers.
Chacun a sa branche, personne ne touche à main, on merge via Pull Request."

Q: "Pourquoi le mot de passe n'est pas dans UserModel ?"

R: "Firebase Auth gère déjà les mots de passe de façon sécurisée.
UserModel ne contient que les données publiques de l'utilisateur.
Le mot de passe ne doit jamais être stocké dans l'app."

Q: "Comment vous testez si tout fonctionne ?"

R: "On teste chacun notre partie avec des Fake implementations en attendant.
Rôle 5 integre tout à la fin et teste la chaîne complète."

SUPPORT & COMMUNICATION

Si quelque chose est bloqué:

1. Posez la question au groupe
2. Pas de solution : contactez Rôle 1 (Lead Backend)
3. Problème grave : tous ensemble

Réunion type (15 min):

- Point de ce qu'on a fait (2 min chacun)
- Blocages ? (3 min)
- Prochaine étape ? (2 min)

Créé: 10 février 2026

Projet: Aidly

Version: 1.0

✓ **Ce document est la source de vérité pour le projet. Consultez-le en cas de doute.**